ASSET REPAIR REQUEST SYSTEM

A PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in Computer Science and Engineering

Submitted by

YASH SHARMA

UID: 24BCS11074

Under the Guidance of

Mr. Jaspreet Singh



Academic Year: 2026–27

BONAFIDE CERTIFICATE

This is to certify that the project report entitled **"Asset Repair Request System"** submitted by **Yash Sharma** (UID: **24BCS11074**) to **Chandigarh University** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Engineering in Computer Science and Engineering** is a Bonafide record of the work carried out by him under my supervision and guidance.

The project was developed on the ServiceNow platform using ServiceNow Studio. The work presented in this report is original and has not been submitted earlier to any other University or Institution for the award of any degree or diploma.

SIGNATURE

Dr. Sanjeev Kumar
(HOD)

University Institute of Engineering

SIGNATURE

Mr. Jaspreet Singh
Technical Trainer

University Institute of Engineering

Submitted for the project viva-voce examination held on

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to everyone who contributed to the successful completion of this project. I am especially thankful to my project guide, **Mr. Jaspreet Singh**, for his valuable guidance, continuous support, and constructive feedback throughout the development of the Asset Repair Request System.

I extend my thanks to the faculty members of the Department of Computer Science and Engineering at Chandigarh University for providing the academic knowledge and learning environment necessary to undertake this project. I also acknowledge ServiceNow for providing the Personal Developer Instance and ServiceNow Studio platform, which enabled the development, configuration, and testing of the application.

Finally, I am grateful to my family and friends for their constant encouragement, understanding, and support throughout this project.

Yash Sharma

UID: 24BCS11074

TABLE OF CONTENTS

List of Figures v

List of Tables vi

Abstract vii

Graphical Abstract viii

List of Abbreviations, Symbols and Nomenclature ix

Chapter 1 Introduction 1

1.1 Project Background and Motivation 1

1.2 Problem Identification and Importance 2

1.3 Existing System and Limitations 3

1.4 Proposed System and Objectives 5

1.5 Scope and Task Identification 5

1.6 Organization of the Report 7

Chapter 2 Literature Survey 8

2.1 Asset Management and Repair-Request Management 8

2.2 IT Service Management Concepts 8

2.3 Workflow Automation and Role-Based Access Control 10

2.4 Review of Existing Approaches 12

2.5 Research Gap and Project Objectives 12

Chapter 3 Design Flow / Process 14

3.1 Concept Generation 14

3.2 Alternative Design Approaches 14

3.3 Selected Design and Justification 15

3.4 Functional and Non-Functional Requirements 15

3.5 System Architecture and Data Flow 15

3.6 Table and Field Design 16

3.7 User Roles and Access-Control Design 16

3.8 Workflow Design 17

3.9 Implementation Process 17

3.10 Constraints and Ethical Considerations 18

Chapter 4 Results Analysis and Validation 19

4.1 Development Environment 19

4.2 Application and Table Implementation 19

4.3 Access-Control and Flow Implementation 22

4.4 Test Methodology and Test Cases 25

4.5 Results and Interpretation 26

4.6 Limitations and Validation 27

Chapter 5 Conclusion and Future Work 28

5.1 Summary of Completed Work 28

5.2 Achievement of Objectives 28

5.3 Conclusions 29

5.4 Limitations 30

5.5 Future Enhancements 31

References 32

Appendix-1 Screenshot Gallery 33

Appendix-2 User Manual 34

Achievements 35

GITHUB REPOSITORY 36

LIST OF FIGURES

Figure 1 Workflow of the Asset Repair Request System viii
Figure 2.1 Comparison of Enterprise ITSM Platforms for Asset Repair.....12
Figure 3.1 Approach A – Formal Multi-Level Approval.....14
Figure 3.2 Approach B – Direct Submission with Automatic Assignment.....15
Figure 3.3 System Architecture of Asset Repair Request System.....16
Figure 3.4 Repair Request Process Flow 17
Figure 4.1 Application Development Environment in ServiceNow Studio.....19
Figure 4.2 Successful Creation of the Asset Repair Request Application.....20
Figure 4.3 Asset Repair Request Table Configuration 20
Figure 4.4 Core Field Configuration of the Asset Repair Request Table 21
Figure 4.5 ARQ Auto-Number Configuration.....21
Figure 4.6 Asset Repair Request Form Layout.....21
Figure 4.7 Application Menu and Module Configuration 22
Figure 4.8 Access Control List Configuration.....23
Figure 4.9 Flow Designer Trigger Configuration 24
Figure 4.10 Automatic Assignment Action Configuration.....24
Figure 4.11 Repair Request Creation.....26
Figure 4.12 Successful Flow Execution.....26
Figure 4.13 Asset Repair Request List View.....27

LIST OF TABLES

Table 1.1 Tasks Identified During Development.....6

Table 2.1 Comparison of Approaches for Managing Repair Requests.....10

Table 3.1 Field Design of Asset Repair Request Table 16

Table 3.2 Role and Permission Matrix.....17

Table 4.1 Test Case Results 25

ABSTRACT

Organizations depend on physical and IT assets such as laptops, monitors and network devices. When these assets are damaged, employees require a structured channel to report the issue, while the Service Desk needs an efficient mechanism to track and resolve repair requests. Informal reporting through email or chat leads to lost requests, unclear ownership and poor visibility.

This project presents the design and implementation of an Asset Repair Request System on the ServiceNow platform using ServiceNow Studio. The application enables employees to raise repair requests and allows the Service Desk to record, assign and track progress until closure. A scoped application was created with three roles (employee, service_desk and admin). The main table Asset Repair Request was configured with ARQ auto-numbering, essential fields (Requested by, Asset, Description, Priority, Status, Assigned to and Work notes), a usable form layout and table-level permissions. A Flow Designer flow named Asset Repair Request – Status and Assignment Automation was implemented to assign newly created requests to the Service Desk automatically.

The application was tested for record creation, auto-number generation, flow execution and basic role access. All primary test cases passed. Known limitations include incomplete verification of employee ownership-based read restriction, unresolved Work notes persistence behaviour, and unsuccessful attempts to publish the application to the ServiceNow App Repository and to link source control. The solution provides a clean, maintainable foundation for asset-repair management within ServiceNow and can be extended with CMDB integration, notifications and portal experiences.

Keywords: ServiceNow, ServiceNow Studio, Asset Repair Request, Flow Designer, Role-Based Access Control, Auto-numbering

GRAPHICAL ABSTRACT

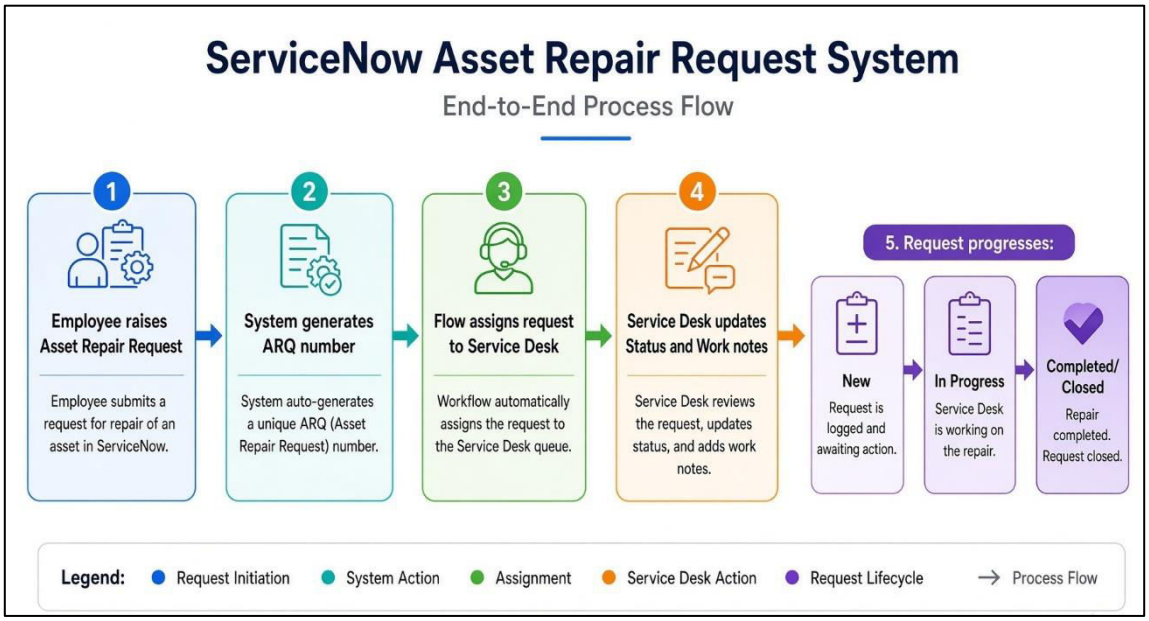


Figure 1: Workflow of the Asset Repair Request System

The process begins when an employee raises an Asset Repair Request. The system automatically generates a unique ARQ number for tracking. A configured flow then assigns the request to the Service Desk. The Service Desk team reviews the request, updates its status, and records relevant work notes. The request progresses through the defined stages of New, In Progress, and finally Completed or Closed.

LIST OF ABBREVIATIONS, SYMBOLS AND NOMENCLATURE

ACL	Access Control List
ARQ	Asset Repair Request (number prefix)
CMDB	Configuration Management Database
ITSM	Information Technology Service Management
PDI	Personal Developer Instance
RBAC	Role-Based Access Control
SLA	Service Level Agreement
UI	User Interface

CHAPTER 1

INTRODUCTION

1.1 Project Background and Motivation

Modern organizations depend on physical and information technology assets to support their daily operations. These assets include laptops, desktop computers, monitors, printers, networking equipment and other devices used by employees. The reliable functioning of these resources is important for maintaining productivity, supporting communication and ensuring the continuity of business activities. When an asset becomes damaged or develops a technical fault, the organization needs a systematic way to report the problem, assign responsibility and monitor the repair process.

In many working environments, asset-related problems are initially reported through emails, telephone calls, instant messages or verbal communication. Although these methods are convenient for individual interactions, they do not always provide a consistent record of the request, its current status, the person responsible for handling it or the actions taken during repair. As the number of employees and assets increases, maintaining an accurate overview through informal communication becomes more difficult.

A structured asset repair request process addresses these issues by maintaining repair information in a centralized system. Each request can be associated with a unique identifier, a requester, an asset, a problem description, a priority and a status. Such information helps the support team identify pending work, distinguish requests and maintain a record of the actions performed. It also provides a clearer basis for reviewing request history and understanding the workload of the support team.

ServiceNow provides a platform for developing business applications and automating organizational workflows. Its application development environment supports the creation of tables, forms, application roles, access controls and workflow actions. These capabilities make it possible to build a focused application without developing every underlying platform component from the beginning.

The Asset Repair Request System was developed as a scoped ServiceNow application to demonstrate how these platform capabilities can be applied to a practical maintenance-related requirement. The application provides a dedicated table for repair requests, uses the ARQ prefix for request numbering, separates employee, Service Desk and administrator responsibilities through application roles, and uses Flow Designer to assign newly created requests to the Service Desk.

The motivation for this project is therefore twofold. From an operational perspective, it demonstrates a more organized method of recording and handling asset repair requests. From a technical and academic perspective, it provides practical experience in scoped application development, table and form configuration, access-control design, workflow automation and functional testing within ServiceNow.

1.2 Problem Identification and Importance

Asset repair activities require a reliable process for reporting faults, recording request details, assigning responsibility and monitoring progress. When these activities are managed through informal communication or disconnected records, important information may become difficult to retrieve and the overall repair process may lack consistency. The Asset Repair Request System addresses this problem by providing a dedicated application for recording and managing repair requests.

The major problems identified for this project are described below.

1. Lack of Centralized Request Records

When repair requests are reported through different communication channels, the information may be distributed across emails, messages or manually maintained records. This makes it difficult to maintain a consolidated view of all requests. A dedicated request table provides a common location for storing details such as the requester, asset, priority, status and description.

2. Difficulty in Identifying Individual Requests

Without a consistent identification mechanism, distinguishing between multiple repair requests can become inconvenient. The proposed system uses automatically generated request numbers with the ARQ prefix. This provides each request with an identifiable reference that can be used when locating and discussing a particular record.

3. Unclear Assignment Responsibility

A repair request must reach the team responsible for handling it. If assignment depends entirely on manual communication, requests may require additional coordination before work can begin. The implemented Flow Designer flow addresses the initial assignment step by assigning newly created requests to the Service Desk. This reduces the delay between request submission and the start of Service Desk activity.

4. Limited Visibility into Request Status

Users and support personnel need to identify whether a request is new, in progress or completed. Recording a status in the request table provides a way to represent the current state of a repair request. The application includes a Status field, although a complete automated status-transition process is outside the currently verified implementation.

5. Need for Controlled Access

Different users require different levels of access. Employees primarily need to submit requests, Service Desk personnel need to handle and update them, and administrators require broader management capabilities. Application roles and Access Control Lists (ACLs) provide a mechanism for defining these permissions. However, the employee ownership-based read restriction was not conclusively verified during testing and remains an area for improvement.

6. Need for Basic Request Validation

Request records should contain the information necessary for processing. Missing asset details, descriptions or other required information can reduce the usefulness of a repair request. The application form includes mandatory fields to encourage complete data entry and applies validation when users submit records.

The importance of addressing these problems lies in establishing a consistent and traceable process for handling repair requests. By combining structured data storage, unique request numbering, role-based permissions and automatic initial assignment, the project demonstrates how a ServiceNow application can support a more organized repair-request workflow. The implemented system provides a foundation for further enhancements, including stronger ownership-based access restrictions, improved work-notes persistence and more comprehensive workflow automation.

1.3 Existing System and Limitations

Before implementing a dedicated Asset Repair Request System, repair-related information may be managed through general communication methods such as emails, telephone calls, instant messages or manually maintained records. These methods can be useful for handling individual requests, particularly in small environments. However, when requests increase in number or involve multiple users and support personnel, maintaining consistent records and monitoring progress can become more challenging.

The following limitations describe the issues that a dedicated repair-request application is intended to address.

1. Fragmented Information Management

When request details are maintained across multiple communication channels, information about the reported fault, affected asset and responsible support personnel may not be available in one consolidated location. A centralized table helps maintain the relevant information within individual request records.

2. Inconsistent Request Identification

Informal reporting methods may not provide a consistent reference number for every repair request. This can make it inconvenient to locate a particular request or distinguish it from similar reports. The proposed application addresses this issue through automatically generated ARQ request numbers.

3. Manual Coordination and Assignment

In a process based primarily on manual communication, a support team may need to identify and assign incoming requests separately. This introduces an additional coordination step. The implemented ServiceNow flow automates the initial assignment of newly created requests to the Service Desk.

4. Limited Structured Status Tracking

Emails and messages can contain updates about repair progress, but they do not necessarily provide a consistent view of the current status of every request. A dedicated Status field allows the application to represent the state of each request in a structured manner.

5. Inconsistent Access Management

When repair information is maintained in general-purpose documents or communication channels, access may be difficult to organize according to the responsibilities of employees, support personnel and administrators. Application roles and ACLs provide a more structured approach to controlling access. Their effectiveness, however, depends on correct configuration and testing.

6. Limited Standardization of Request Information

Requests submitted without a consistent format may omit important details, such as the affected asset or a description of the problem. A dedicated form with defined fields and mandatory-field validation encourages users to provide the information required for processing.

The Asset Repair Request System was designed to address these limitations through centralized storage, automatic numbering, structured forms, role-based access and initial assignment automation. Ownership-based read restrictions and Work notes persistence remained unresolved during testing.

1.4 Proposed System and Objectives

The proposed system is a scoped ServiceNow application named Asset Repair Request System. Its primary objectives are:

1. To enable employees to create and view their own asset repair requests.
2. To enable Service Desk agents to view, update and track all repair requests.
3. To generate human-readable request numbers with the prefix ARQ.
4. To enforce role-based access control using three application roles.
5. To automate initial assignment of new requests to the Service Desk via Flow Designer.
6. To provide a clean form layout and consistent status values for tracking progress.

1.5 Scope and Task Identification

The scope of the Asset Repair Request System covers the development of a basic repair-request management application on the ServiceNow platform for submitting, storing and managing repair requests. It also includes role configuration, access control, and initial workflow automation for assignment.

1.5.1 Project Scope

The project includes the following functional and technical areas.

1. Repair Request Management

The application provides a dedicated table and form for storing repair-request information. The configured fields include the request number, requested-by user, asset, assigned user, priority, status, description and work notes. This structure supports consistent data entry and later tracking by the Service Desk.

2. Automatic Request Numbering

The application uses the ARQ prefix to generate identifiable request numbers. This allows individual repair requests to be referenced consistently when viewing or managing records.

3. Form and Field Configuration

The request form is configured to display the relevant fields for data entry and record management. Mandatory fields help ensure that essential information is provided when a request is submitted.

4. Role-Based Access Configuration

The application defines Employee, Service Desk and Admin roles. ACLs are configured to distinguish the operations available to these roles. Further verification is required for the employee ownership-based read restriction.

5. Workflow Automation

Flow Designer is used to assign newly created repair requests to the Service Desk automatically. The current scope does not include a fully automated status-transition lifecycle.

6. Application Navigation

An application menu and module provide an entry point for accessing the repair-request records within the ServiceNow instance.

7. Functional Testing

Testing covers request creation, record saving, numbering, administrative operations and flow execution. Unresolved issues such as Work notes persistence are also documented.

1.5.2 Tasks Identified During Development

The development work was divided into a sequence of implementation tasks.

Table 1.1 Tasks Identified During Development

Task	Description
Application setup	Create the scoped application and define its roles.
Table design	Create the Asset Repair Request table and configure its fields.
Number configuration	Configure automatic request numbering with the ARQ prefix.
Form design	Arrange the form fields and configure mandatory-field requirements.
Access control	Create and configure ACLs for the application roles.
Workflow development	Configure the record-created trigger and automatic assignment action.
Navigation setup	Create the application menu and repair-request module.
Functional testing	Test record creation, saving, administrative operations and flow execution.
Documentation	Record the implementation process, test evidence, limitations and future improvements.

1.5.3 Limitations and Exclusions from the Current Scope

The project demonstrates a basic repair-request workflow rather than a complete enterprise asset-management solution. Advanced capabilities such as automated notifications, service-level agreement monitoring, reporting dashboards, integration with external asset databases and a fully automated status lifecycle are not established as completed features.

Publishing the application to the ServiceNow App Repository was unsuccessful because of a vendor-information or connectivity error. The attempted source-control integration also encountered a compatibility error. In addition, the employee ownership-based read restriction and Work notes persistence require further investigation.

These exclusions define the current implementation boundary and provide a basis for identifying future development priorities.

1.6 Organization of the Report

Chapter 2 reviews relevant literature and platform capabilities. Chapter 3 describes the design process, requirements, architecture, data model, security and workflow. Chapter 4 presents implementation details, test cases and results. Chapter 5 summarizes achievements, limitations and future enhancements.

CHAPTER 2

LITERATURE SURVEY

2.1 Asset Management and Repair-Request Management

A literature survey examines existing systems, technologies and approaches related to the problem being addressed by a project. It helps identify established practices, understand the capabilities of available platforms and determine areas where a proposed solution can provide practical value. The survey also supports the justification of design choices made later in the project.

The Asset Repair Request System focuses on organizing repair requests through a centralized application. Its development involves several important concepts, including service request management, workflow automation, role-based access control, database-backed record management and application development platforms.

Traditional repair-request processes may rely on emails, telephone calls, spreadsheets or manually maintained records. Although these methods can be suitable for small-scale operations, they may create difficulties in maintaining consistent records, identifying request ownership and tracking assignments. A centralized digital system provides a structured approach to storing request information and managing access to records.

The project uses ServiceNow as its development platform. ServiceNow provides application development capabilities that support the configuration of tables, forms, access controls and automated workflows. These capabilities are relevant to the project's objective of creating a structured repair-request management application. The platform also offers a Personal Developer Instance suitable for academic prototyping and testing.

This chapter examines the concepts and technologies associated with the project, compares relevant approaches and identifies the requirements that influenced the proposed system design.

2.2 IT Service Management Concepts

Repair requests can be managed through several approaches, depending on the size of an organization, the number of assets involved and the complexity of its maintenance activities. Common approaches include manual record keeping, spreadsheet-based tracking, email-based communication and centralized service management applications.

2.2.1 Manual Record-Keeping Systems

In a manual system, repair requests are recorded in registers, paper forms or other physical documents. The responsible staff members review these records and coordinate the repair process.

This approach is straightforward to establish and does not require specialized software. However, searching for older requests, updating records and maintaining consistent information can become difficult as the number of requests increases. Physical records may also be misplaced or duplicated.

2.2.2 Spreadsheet-Based Systems

Spreadsheets provide a digital method for recording repair requests. Information such as asset details, request dates, priorities and request statuses can be maintained in rows and columns.

Compared with paper-based records, spreadsheets make searching, sorting and basic reporting easier. Nevertheless, simultaneous updates, access management and consistent assignment of responsibilities may require additional controls. A spreadsheet alone does not necessarily provide a complete request-management workflow.

2.2.3 Email and Communication-Based Systems

Organizations may receive repair requests through emails, telephone calls or messaging platforms. Employees communicate the problem to the responsible team, which then coordinates the required work.

This approach supports direct communication and allows users to describe problems in detail. However, request information may remain distributed across multiple conversations. Identifying the latest status, locating previous communication and determining responsibility can require manual effort.

2.2.4 Centralized Service Management Applications

Centralized service management applications store requests in structured records and provide interfaces for submitting, reviewing and updating them. Depending on their configuration, these systems can support unique request identifiers, controlled access, assignment rules and workflow automation.

Such applications provide a more organized approach to managing requests because information can be maintained in a common system. Their effectiveness depends on appropriate data design, access configuration, workflow logic and testing.

The proposed Asset Repair Request System follows this centralized approach. It uses a dedicated ServiceNow table, a configured request form, ARQ-prefixed request numbering, role-based access controls and a Flow Designer workflow that assigns newly created requests to the Service Desk.

2.2.5 Comparative Summary

Table 2.1 Comparison of Approaches for Managing Repair Requests

Approach	Main advantage	Main limitation
Manual records	Simple to establish	Difficult to search and maintain at scale
Spreadsheets	Easy sorting and basic reporting	Limited built-in workflow and access controls
Email and messaging	Convenient communication	Information may remain distributed
Centralized application	Structured records and configurable workflows	Requires configuration, testing and maintenance

The comparison indicates that a centralized application is suitable for the project's objective of organizing repair requests. The ServiceNow implementation applies this approach while retaining a defined scope and acknowledging the limitations that require further validation.

2.3 Workflow Automation and Role-Based Access Control

ServiceNow is a cloud-based platform designed to develop applications that support organizational processes and data management. It provides tools for configuring data tables, user interfaces, access permissions, and workflow automation within a unified environment. App Engine Studio simplifies application development by enabling developers to manage application data, user interfaces, business logic, automated workflows, and security settings in one place. It also supports the development of applications tailored to specific organizational requirements, reducing the need to build every component from scratch. These capabilities help maintain consistency across application components and make future modifications and maintenance more manageable. Further details are available in the official ServiceNow App Engine Studio documentation.

2.3.1 Scoped Application Development

A scoped application groups application components within a defined application scope. This helps organize custom development and distinguish application components from those belonging to other applications.

In this project, the Asset Repair Request System was created as a scoped application. Its components include the repair-request table, configured roles, access-control rules, application navigation and workflow automation. The scoped structure provides a dedicated environment for developing and maintaining the project's functionality.

2.3.2 Tables and Forms

Tables are used to store application records, while fields define the information maintained for each record. Forms provide an interface through which users can enter, view and update that information.

The project uses a dedicated Asset Repair Request table containing fields such as Number, Requested by, Asset, Assigned to, Priority, Status, Description and Work notes. The request number uses the ARQ prefix to provide a recognizable identifier. The form layout groups the relevant fields into a practical interface for creating and managing repair requests.

2.3.3 Roles and Access Controls

Access control determines which users can access records and perform operations. ServiceNow uses roles and Access Control Lists (ACLs) to manage permissions for reading, creating, updating, and deleting records.

In the Asset Repair Request System, three roles have been defined: Employee, Service Desk, and Admin. Separate ACL rules have been configured to provide different levels of access according to each role's responsibilities. These permissions are intended to support controlled request submission, record management, and administrative operations. However, the employee-specific restriction that should limit read access to requests associated with the employee has not yet been fully verified. Therefore, ownership-based record isolation remains an area requiring further testing.

2.3.4 Flow Designer and Workflow Automation

ServiceNow Flow Designer enables automated workflows through a visual interface. A trigger starts the workflow when an event occurs, and actions define the operations that follow.

In the Asset Repair Request System, a flow runs when a new repair request is created and assigns it to the Service Desk user. The flow was tested and activated.

The current workflow focuses on automating the initial assignment of repair requests. It does not automatically manage the complete status lifecycle of a request. Additional workflow logic would be required to support automated status changes and other advanced service-management operations.

2.3.5 Relevance to the Proposed System

ServiceNow provides the core capabilities for this project in one platform. Scoped applications organize components, tables and forms support structured requests, and Flow Designer handles initial assignment.

These capabilities make ServiceNow suitable for demonstrating a basic repair-request management process. The reliability of the resulting application still depends on appropriate configuration and testing, particularly for access restrictions and record updates.

2.4 Review of Existing Approaches

Enterprise ITSM platforms such as ServiceNow, BMC Helix and Jira Service Management offer incident and request modules that can be adapted for asset repair. Low-code platforms reduce development time compared with traditional custom development.

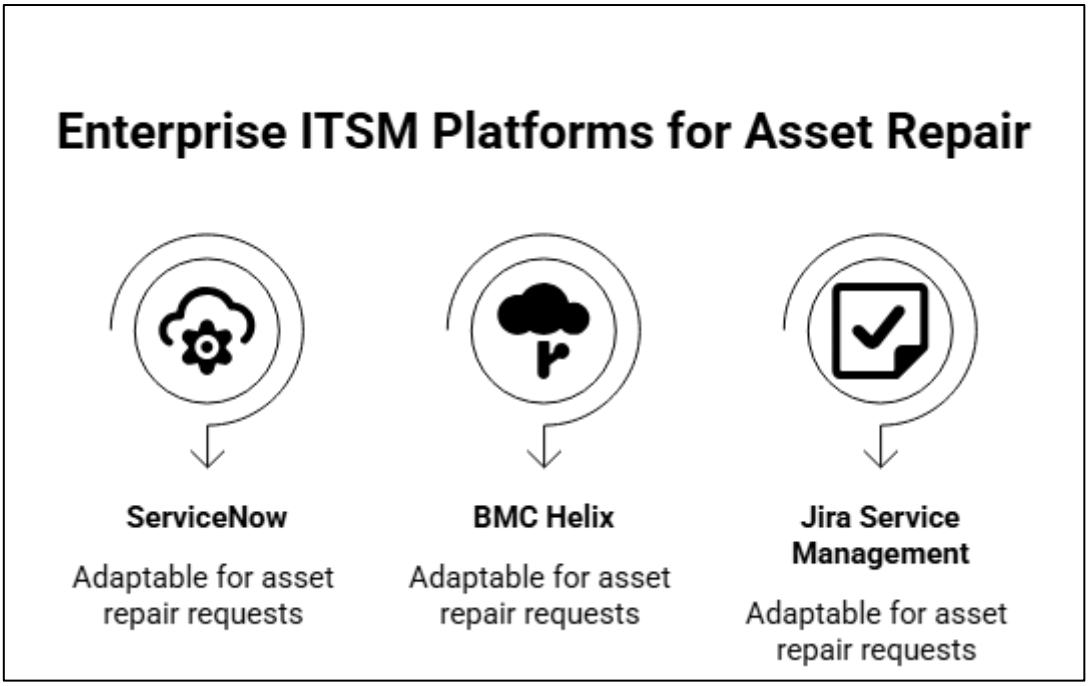


Figure 2.1 Comparison of Enterprise ITSM Platforms for Asset Repair

2.5 Research Gap and Project Objectives

Although general IT Service Management (ITSM) platforms support the recording and management of asset-related issues, a lightweight application specifically designed for repair-request handling can provide a more focused approach. Such an application can organize request information, assign identifiable request numbers, maintain a simplified status structure, and support the allocation of requests to the Service Desk team.

The Asset Repair Request System addresses this practical need by providing a dedicated application developed on the ServiceNow platform. It uses a structured table and form to maintain repair-request information in a centralized location. The ARQ numbering scheme helps identify individual requests, while role-based access controls are configured to distinguish user responsibilities. Flow Designer is used to automate the initial assignment of newly created requests to the designated Service Desk user. These features demonstrate how ServiceNow's application development capabilities can be applied to a specific operational requirement. The project therefore presents a focused implementation of repair-request management while identifying areas, such as ownership-based access restrictions and extended workflow automation, that require further validation or development.

CHAPTER 3

DESIGN FLOW / PROCESS

3.1 Concept Generation

The initial concept was a simple request form in which an employee selects or names the damaged asset, describes the problem, sets a priority and submits the request. The Service Desk would then receive the request, update its status and record work notes until closure. Automation was limited to number generation and initial assignment, keeping the design reliable and easy to test.

3.2 Alternative Design Approaches

Two principal alternatives were considered:

- Approach A – Formal multi-level approval before any work begins (employee → manager → Service Desk).

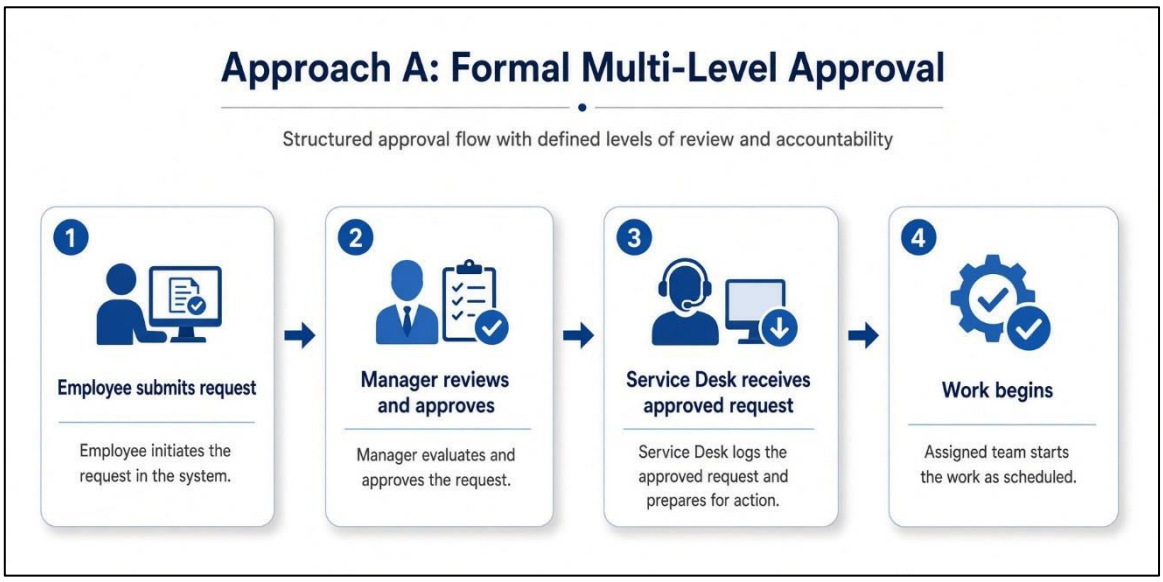


Figure 3.1 Approach A – Formal Multi-Level Approval

- Approach B – Direct submission with automatic assignment to Service Desk and status-based tracking (New → In Progress → On Hold → Completed → Closed).

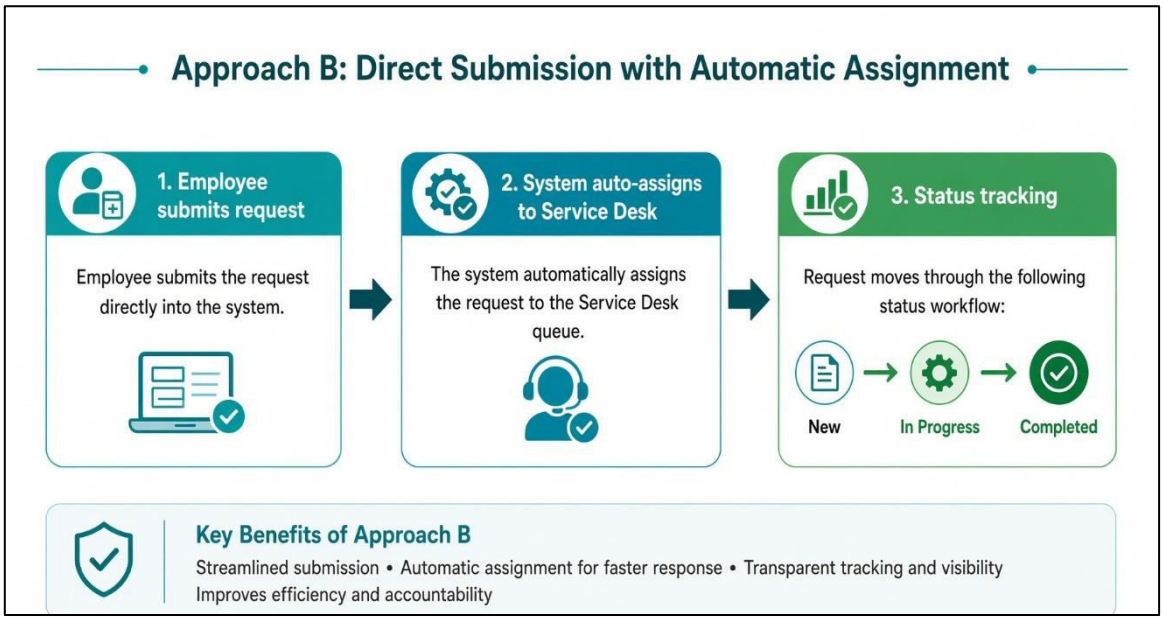


Figure 3.2 Approach B – Direct Submission with Automatic Assignment

Approach A provides stronger managerial control but increases process complexity, requires additional roles and lengthens the time before repair work can start. Approach B is faster for the requester and simpler to implement and test.

3.3 Selected Design and Justification

Approach B was selected. It matches the original project description ("Employees report damaged company assets. IT records repair requests and tracks progress"), minimizes moving parts, and still demonstrates roles, table design, auto-numbering, form layout, security and Flow automation. The reduced complexity improves reliability for an academic demonstration while leaving room for later addition of approvals if required.

3.4 Functional and Non-Functional Requirements

Functional requirements include: employee creation and viewing of own requests; Service Desk viewing and updating of all requests; automatic ARQ numbering; capture of Asset, Description, Priority and Status; automatic assignment on creation; and administrator full access.

Non-functional requirements emphasize usability of the form, strict role-based security, consistent auto-numbering, maintainable scoped-application structure, and the ability to extend the design later. These qualities ensure the system remains practical for users while supporting future enhancements without major redesign.

3.5 System Architecture and Data Flow

The architecture consists of five logical layers: Data (Asset Repair Request table), Security (roles and ACLs), Presentation (form), Automation (Flow Designer) and Navigation (application module). Data flow is straightforward: an authenticated employee creates a record; the platform generates the ARQ number and the Flow sets the Assigned to field; the Service Desk subsequently updates Status and Work notes until the request is closed.

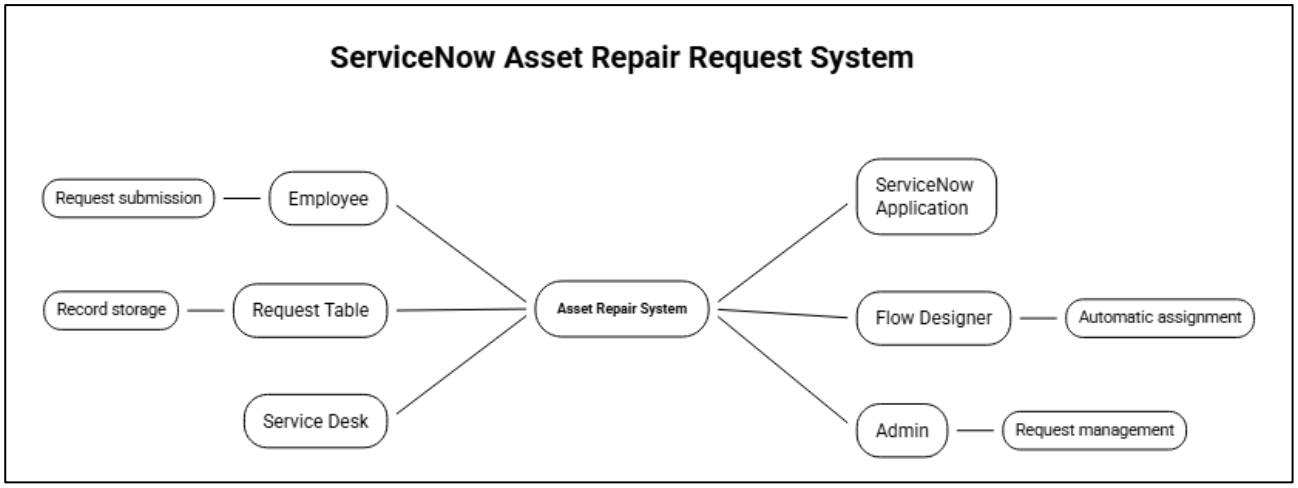


Figure 3.3 System Architecture of Asset Repair Request System

3.6 Table and Field Design

The main table is labelled Asset Repair Request (technical name asset_repair_request within the application scope). Auto-numbering is configured with prefix ARQ, starting number 1000 and seven digits, producing values such as ARQ0001001. This numbering scheme provides a clear and sequential identifier for every repair request created in the system. The seven-digit format also allows sufficient capacity for growth without requiring future changes to the numbering logic.

Table 3.1 Field Design of Asset Repair Request Table

Field Label	Type	Mandatory	Notes
Number	String	System	ARQ auto-generated
Requested by	Reference (User)	Yes	Requester
Asset	String / Reference	Yes	Damaged asset
Description	String (4000)	Yes	Problem description
Priority	Choice	Yes	1-Critical to 4-Low
Status	Choice	Yes	New (default), In Progress, On Hold, Completed, Closed
Assigned to	Reference (User)	No	Service Desk assignee
Work notes	Journal / String	No	Internal notes

3.7 User Roles and Access-Control Design

Table 3.2 Role and Permission Matrix

Role	Responsibility	Access
employee	Creates and views own requests	Create + Read
service_desk	Handles and updates all requests	Create + Read + Write
admin	Full application administration	Full access

Table-level permissions were set at creation time. Additional ACLs were configured; however, conclusive verification that employees are restricted to their own records was not completed during the available testing window.

3.8 Workflow Design

A single Flow named "Asset Repair Request – Status and Assignment Automation" was designed to streamline the initial handling of new requests. The trigger is "Record Created" on the Asset Repair Request table, ensuring the automation runs immediately after an employee submits a request. The sole action is an Update Record that sets the Assigned to field to the designated Service Desk user, thereby eliminating the need for manual assignment at the point of creation. The flow does not automate status transitions; status changes remain a manual responsibility of the Service Desk so that agents can assess priority and progress before updating the record. This limited scope keeps the automation simple, reliable, and easy to maintain while still reducing routine administrative effort.

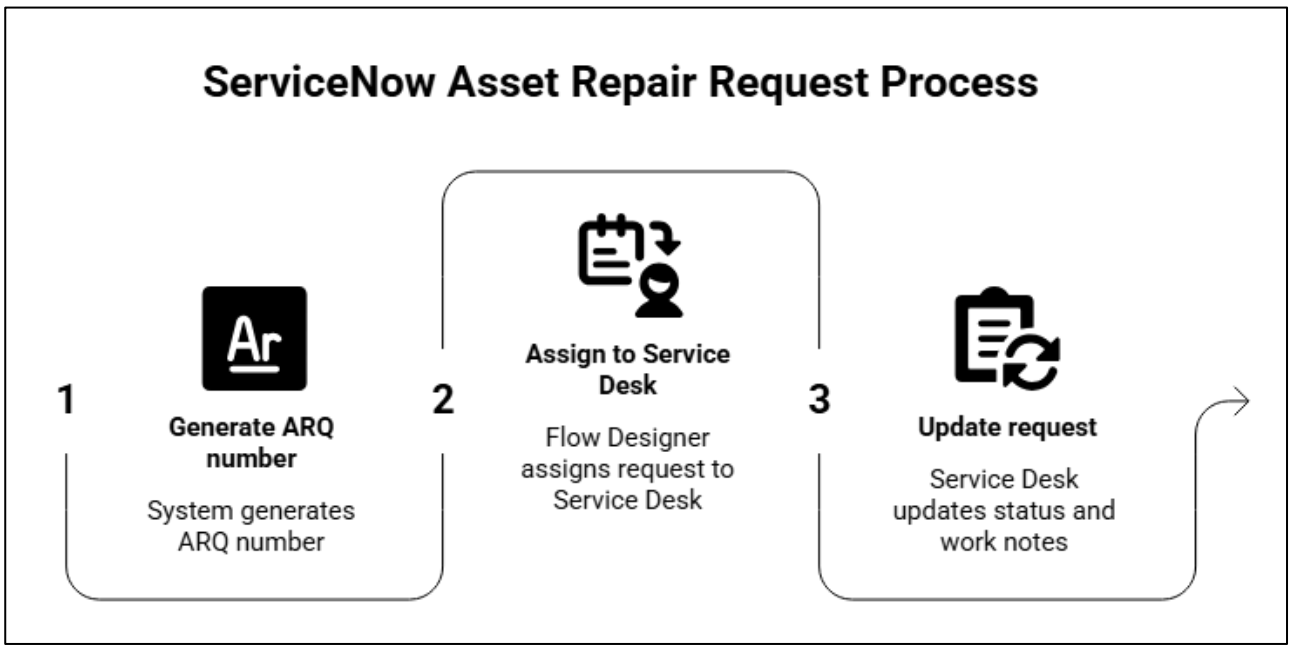


Figure 3.4 Repair Request Process Flow

3.9 Implementation Process

The implementation of the Asset Repair Request System was completed through a sequence of development activities on the ServiceNow platform. The process began with creating a scoped application and defining the roles required for different categories of users. A dedicated Asset Repair Request table was then created, and its fields were configured to store the information required for managing repair requests. Automatic numbering was configured using the ARQ prefix to provide identifiable request numbers.

After establishing the data structure, the request form was arranged for data entry. Access Control Lists were configured for the Employee, Service Desk, and Admin roles, and a Flow Designer workflow was created to assign new requests. The application menu and module were added, and functional tests were performed to examine record creation, saving, and workflow execution.

3.10 Constraints and Ethical Considerations

The application was developed and tested in a ServiceNow Personal Developer Instance (PDI). This environment supports learning and prototyping but differs from production in governance, deployment, and scale. The results therefore demonstrate configured functionality within the development instance only.

Ethical considerations were addressed by avoiding the use of personal or sensitive organizational data during development and testing. The application uses the standard ServiceNow User reference to associate requests with users rather than introducing unnecessary personal-information fields. This approach keeps the information collected relevant to the purpose of repair-request management.

The application also incorporates separate Employee, Service Desk, and Admin roles to support the principle of least privilege, under which users should receive only the permissions needed for their responsibilities. However, the effectiveness of this approach depends on the correct configuration and verification of the associated ACLs. In particular, the employee ownership-based read restriction requires further validation before complete record-level isolation can be claimed.

The project also recognizes the importance of accurate testing and transparent reporting. Features that have been implemented are distinguished from those that remain incomplete or require further investigation. These considerations help ensure that the documented results reflect the actual state of the application and provide a responsible foundation for future improvements.

CHAPTER 4

RESULTS ANALYSIS AND VALIDATION

4.1 Development Environment

The application was built on a ServiceNow Personal Developer Instance using ServiceNow Studio. The browser-based studio provided guided creation of the application, tables, fields, forms, security artefacts and Flow Designer flows. No additional local development tools were required.

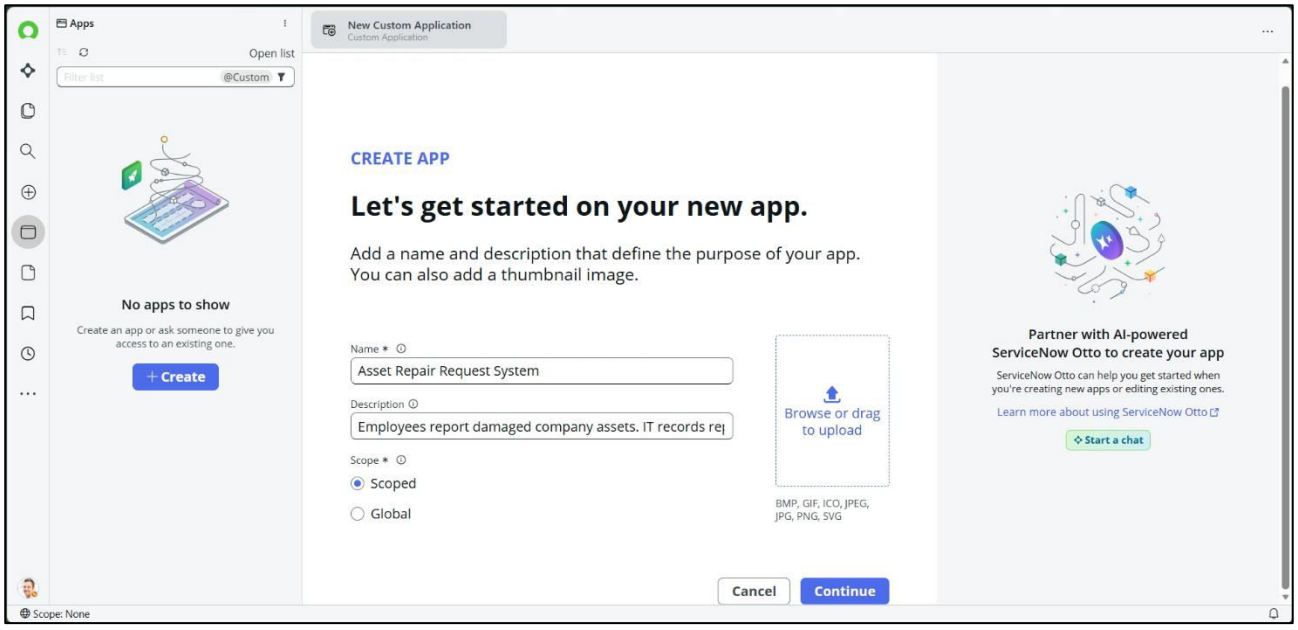


Figure 4.1 Application Development Environment in ServiceNow Studio

4.2 Application and Table Implementation

The scoped application Asset Repair Request System was created successfully within ServiceNow Studio. The table Asset Repair Request was defined with auto-numbering (prefix ARQ, start 1000, seven digits), ensuring each request receives a unique and human-readable identifier. All planned fields were added and configured with appropriate data types, mandatory settings, and reference relationships where required. The form was carefully arranged for logical data entry, grouping related fields and placing the most frequently used information in a clear sequence. An application module was created to expose the list and form in the navigator, allowing authorized users to access the functionality directly from the ServiceNow application menu. Role-based permissions were applied at the table level so that employees, Service Desk agents, and administrators receive the correct level

of access. The overall structure was validated to confirm that records could be created, viewed, and updated according to the design specifications. These implementation steps established a solid foundation for the subsequent workflow and testing activities.

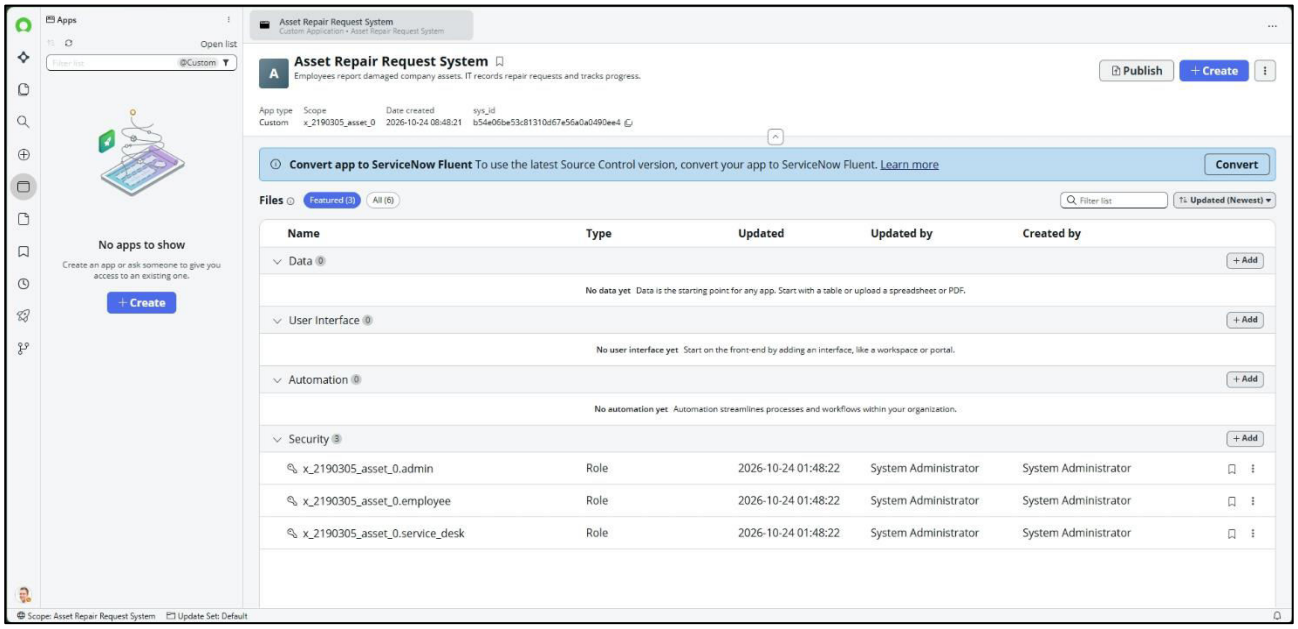


Figure 4.2 Successful Creation of the Asset Repair Request Application

The table was configured with automatic request numbering using the ARQ prefix. The form layout was arranged to support structured request entry and display the relevant information to users. An application menu and module were also created to provide access to the repair request list.

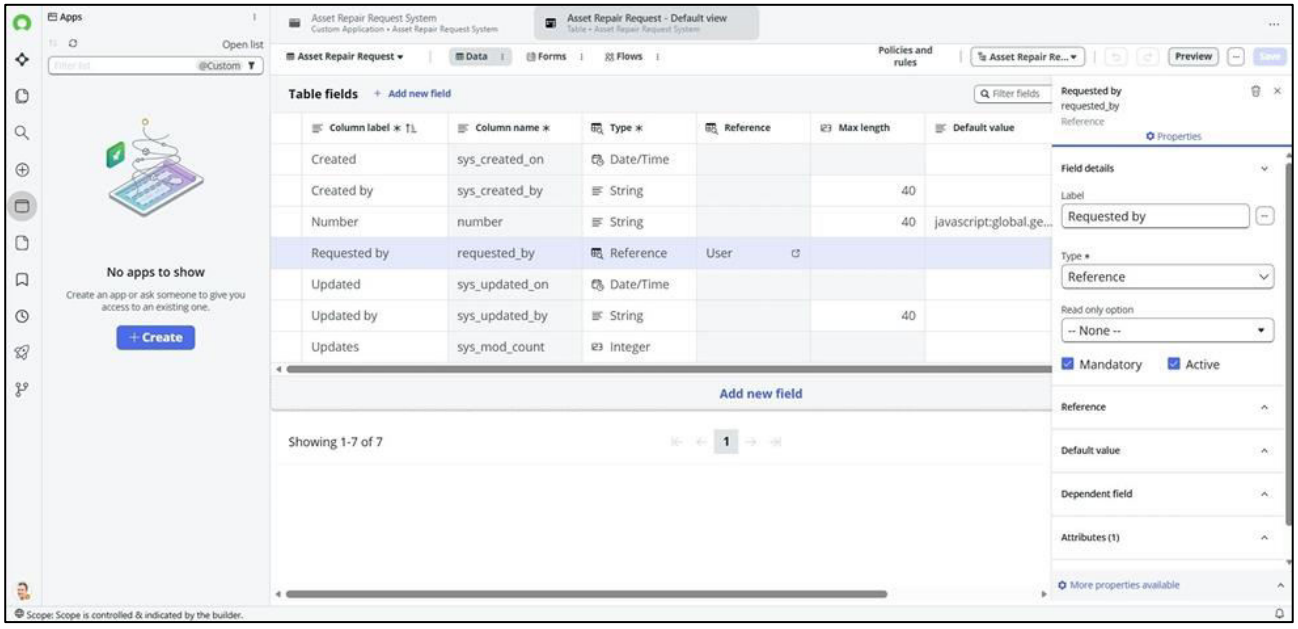


Figure 4.3 Asset Repair Request Table Configuration

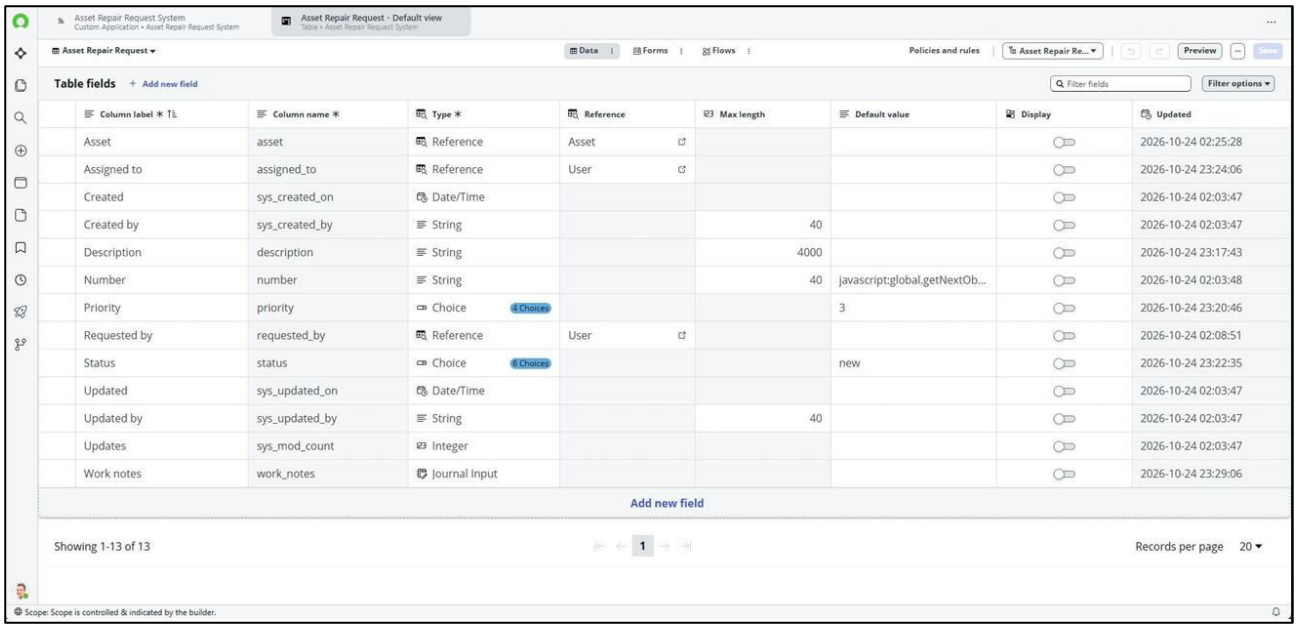


Figure 4.4 Core Field Configuration of the Asset Repair Request Table

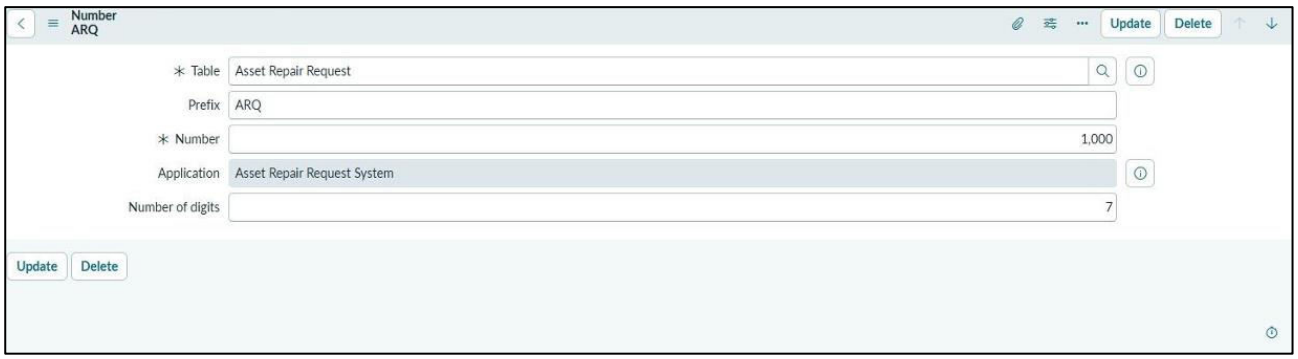


Figure 4.5 ARQ Auto-Number Configuration

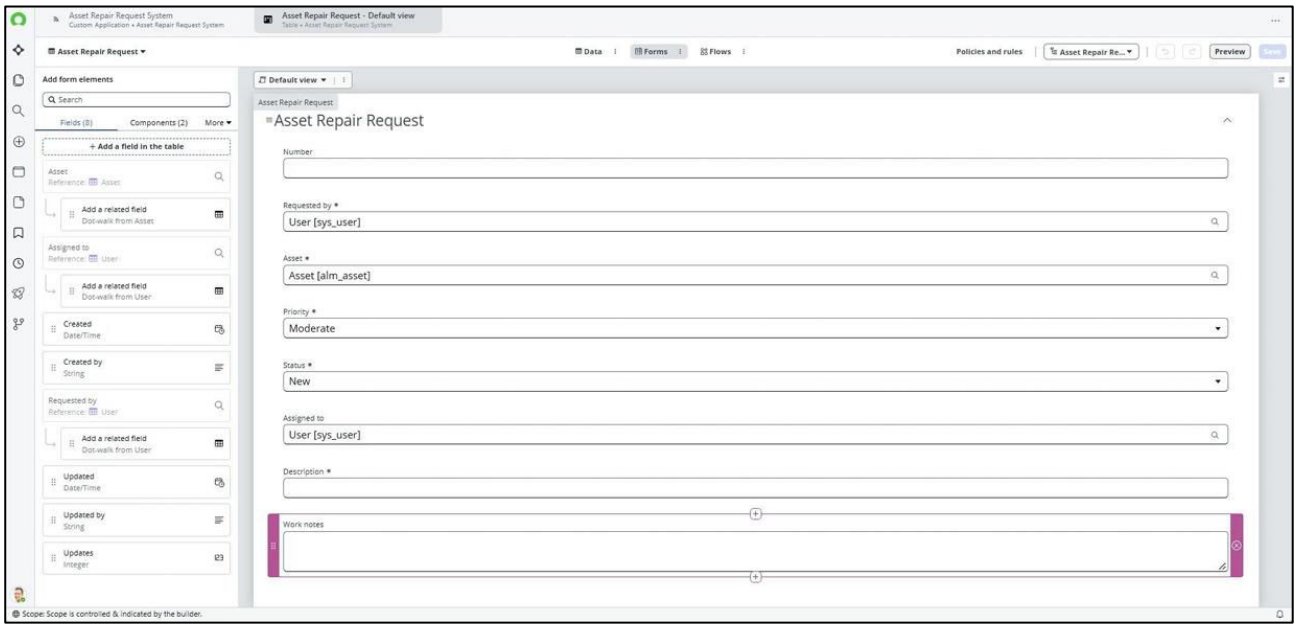


Figure 4.6 Asset Repair Request Form Layout

The application menu and module provide a navigation entry point for accessing repair requests within the ServiceNow instance.

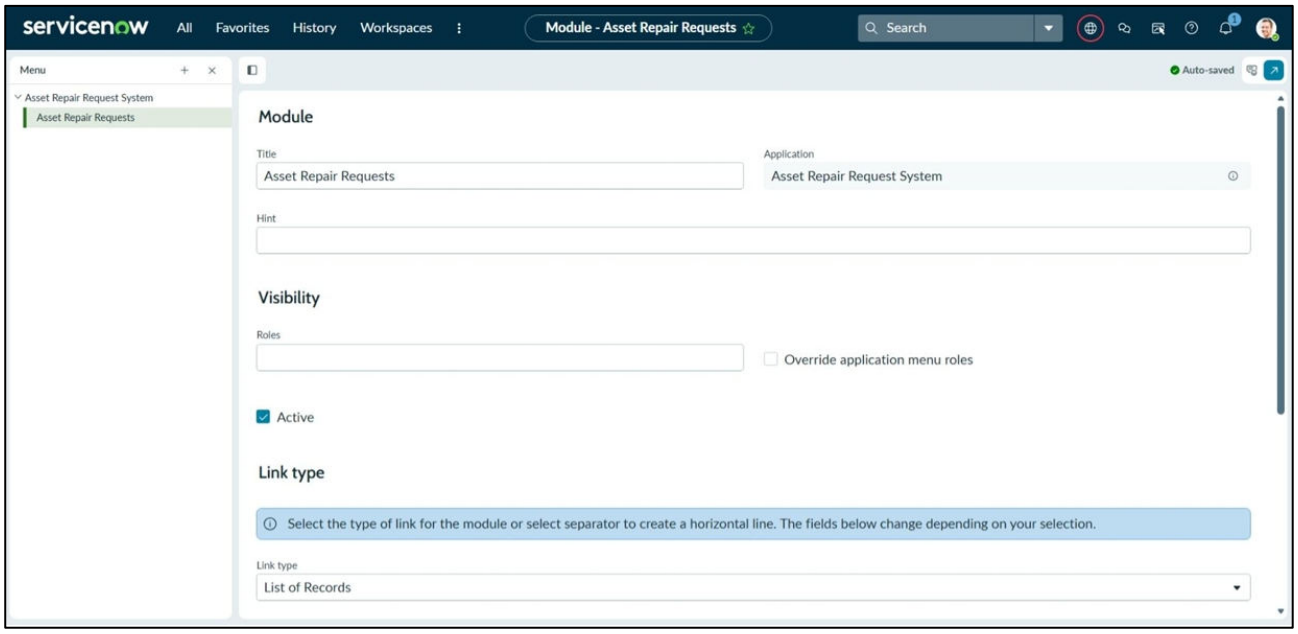


Figure 4.7 Application Menu and Module Configuration

4.3 Access-Control and Flow Implementation

The application defines separate roles for employees, Service Desk personnel, and administrators. These roles are intended to separate request submission, operational handling, and administrative management. Access Control Lists (ACLs) were configured to control the operations available to each role. Employees are intended to create and view their own repair requests, while Service Desk personnel can update request statuses and work notes for all requests. Administrators are configured with broader permissions for application and record management. However, the employee ownership-based read restriction has not been conclusively verified during testing and requires further validation. Therefore, the application's role-based access configuration is in place, but complete record-level isolation for employees cannot yet be confirmed.

4.3.1 Role-Based Access Control

The configured ACLs provide role-based access to the Asset Repair Request table. Employees are assigned request creation and read permissions, Service Desk personnel have permissions for request handling and updates, and administrators are configured with broader management permissions. These access rules help separate responsibilities among different categories of users. They are intended to prevent unauthorized operations and ensure that users can access functions according to their assigned roles. The ACL configuration forms an important part of the application's security design, although ownership-based read restrictions for employees require further verification.

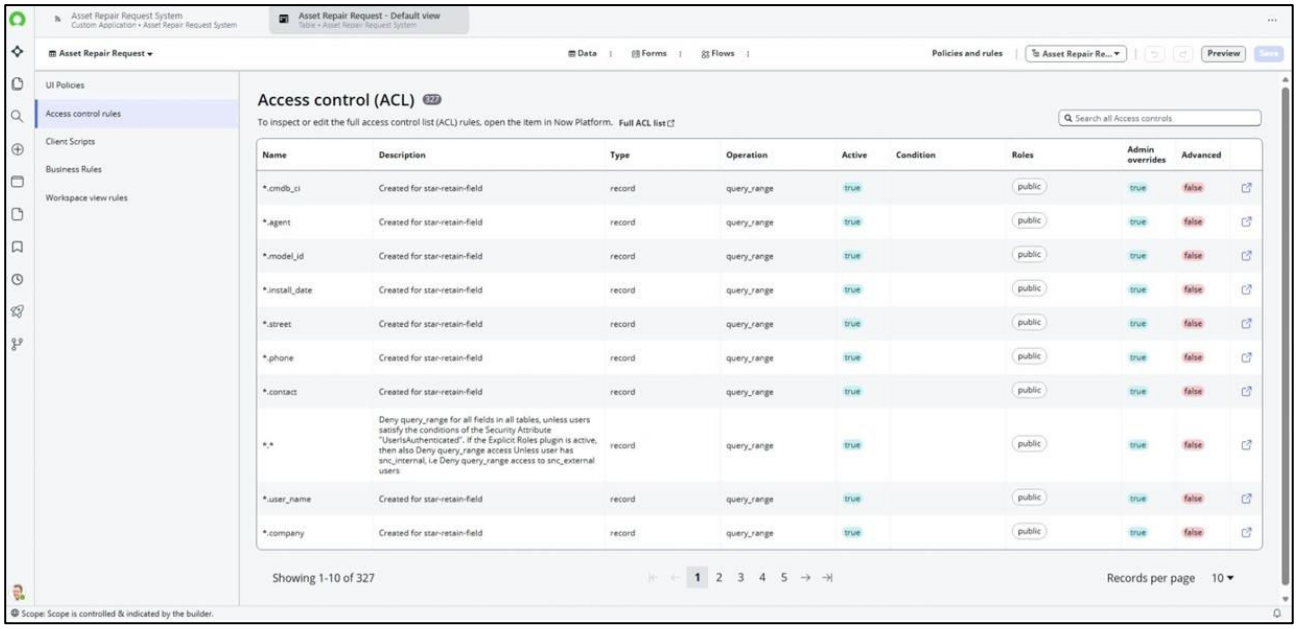


Figure 4.8 Access Control List Configuration

The employee ownership restriction requires further verification. Therefore, the implementation should not be described as conclusively enforcing access to only the employee's own requests.

4.3.2 Flow Designer Trigger Configuration

A Flow Designer flow named Asset Repair Request - Status and Assignment Automation was created for the Asset Repair Request table to handle the initial processing of new submissions. The flow uses a record-created trigger, allowing the configured action to execute immediately when a new repair request is created by an employee. This design ensures that every newly submitted request is captured and processed without requiring any manual intervention at the point of entry. By automating the first step of the workflow, the system reduces the time between request submission and the start of Service Desk activity. The flow was built and tested within ServiceNow Studio, confirming that the trigger correctly identifies new records and initiates the assigned action. The Update Record action sets the Assigned to field to the designated Service Desk user, providing a clear ownership point for every new request. Testing confirmed that the flow executes consistently for records created through the form. The limited scope of the flow—covering only initial assignment—keeps the automation simple and reliable for the academic demonstration. No status-transition actions were included, so the Service Desk retains control over subsequent progress updates. Overall, the implementation provides a reliable and consistent mechanism for starting the repair request lifecycle and establishes a foundation for any future workflow extensions.

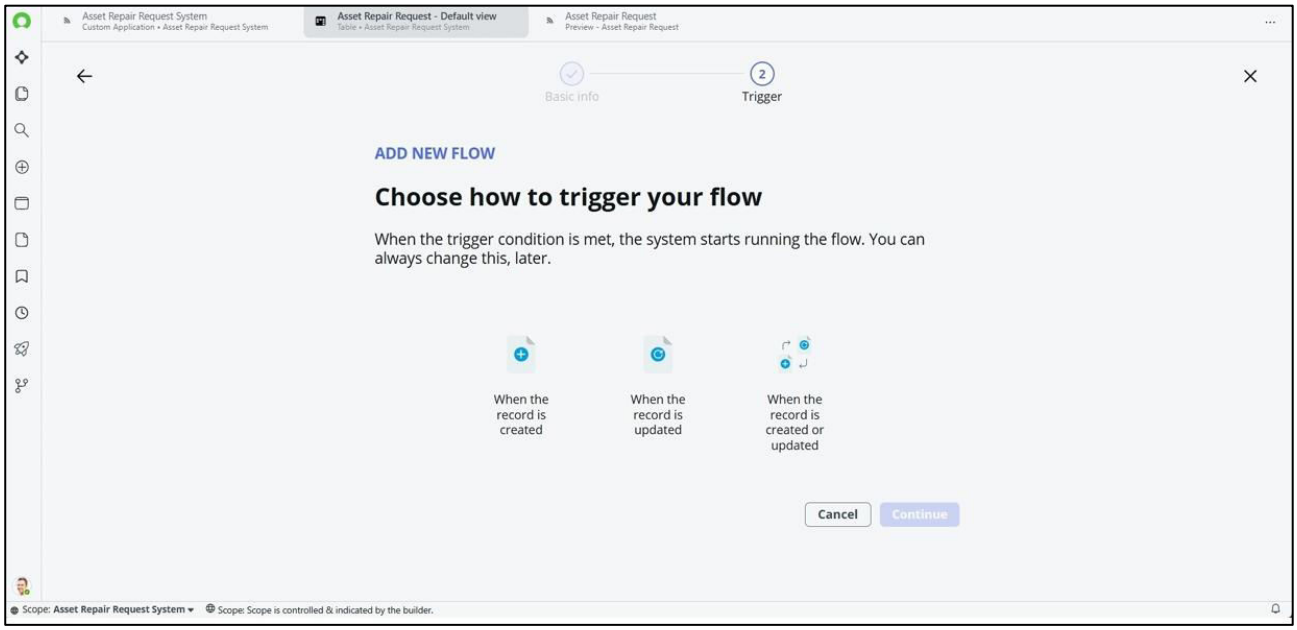


Figure 4.9 Flow Designer Trigger Configuration

4.3.3 Automatic Assignment Action

An Update Record action was configured to assign newly created repair requests to the Service Desk user. This reduces the need for manual assignment at the time of request creation.

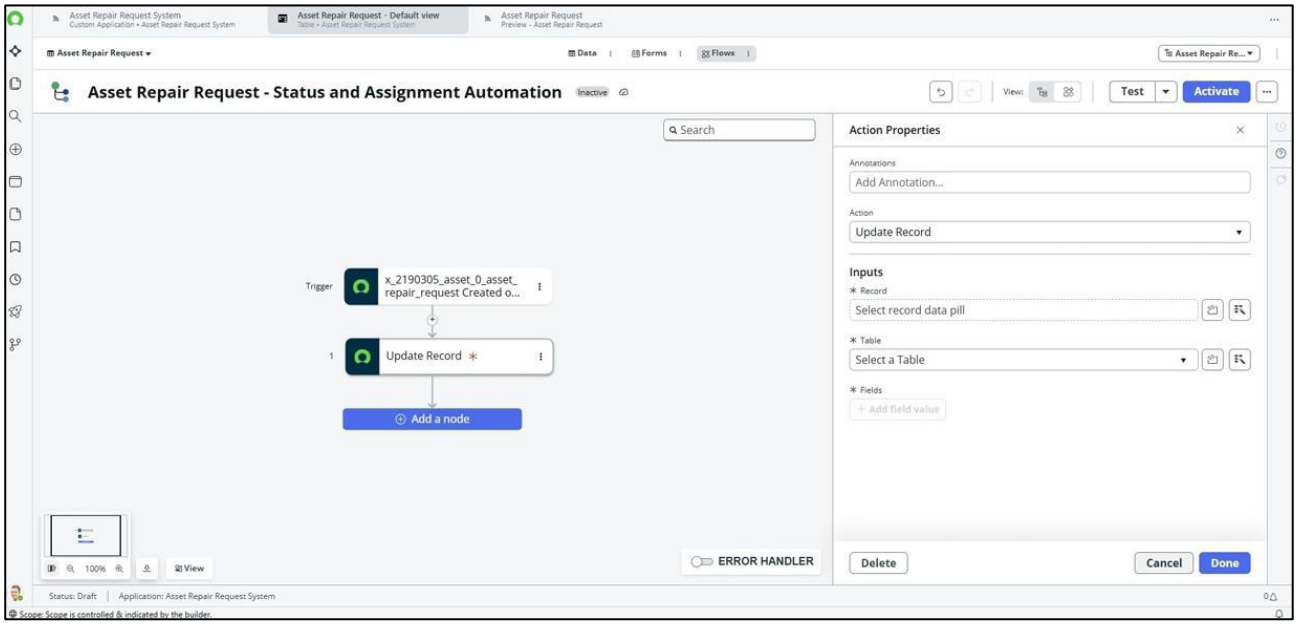


Figure 4.10 Automatic Assignment Action Configuration

The implemented flow demonstrates automatic assignment on record creation. It should not be interpreted as a complete automated status-transition workflow, because separate status-transition

actions have not been established. This distinction is important for understanding the current scope of automation and the remaining manual responsibilities of the Service Desk.

4.4 Test Methodology and Test Cases

Testing was performed to check request creation, automatic numbering, default values, mandatory-field validation, workflow execution, and administrative operations within the ServiceNow instance. A new repair request was submitted through the application form to confirm that the required fields could be completed and that the record could be saved successfully. The generated ARQ number was examined to verify that the auto-numbering configuration produced the expected format. Default values for Status and other fields were observed to ensure they appeared as designed. Mandatory-field validation was checked by attempting to submit the form with required fields left empty. The Flow Designer flow was executed and its result reviewed to confirm that the Assigned to field was updated correctly. Administrative operations, including opening the list view and updating an existing record, were also exercised. These checks provided evidence that the core functions of the application operate as intended within the development environment.

4.4.1 Test Case Design

The test cases were selected to cover the main application functions and relevant validation checks. The expected outcomes were compared with the observed results during testing.

Table 4.1 Test Case Results

TC ID	Description	Expected	Actual	Status
TC-01	Create request with mandatory fields	Record saved with ARQ number	ARQ0001001 created	Pass
TC-02	Auto-number generation	Number starts with ARQ	ARQ prefix confirmed	Pass
TC-03	Default Status value	Status = New	New observed	Pass
TC-04	Flow auto-assignment	Assigned to populated	Flow completed	Pass
TC-05	Mandatory field validation	Save blocked if empty	Validation enforced	Pass
TC-06	Service Desk update	Status/Work notes updatable	Update succeeded	Partially verified
TC-07	Admin delete	Record deleted	Delete succeeded	Pass

4.4.2 Request Creation and Validation

A repair request was created through the application form to verify that information could be entered and saved, and the generated request number and mandatory-field behaviour were checked.

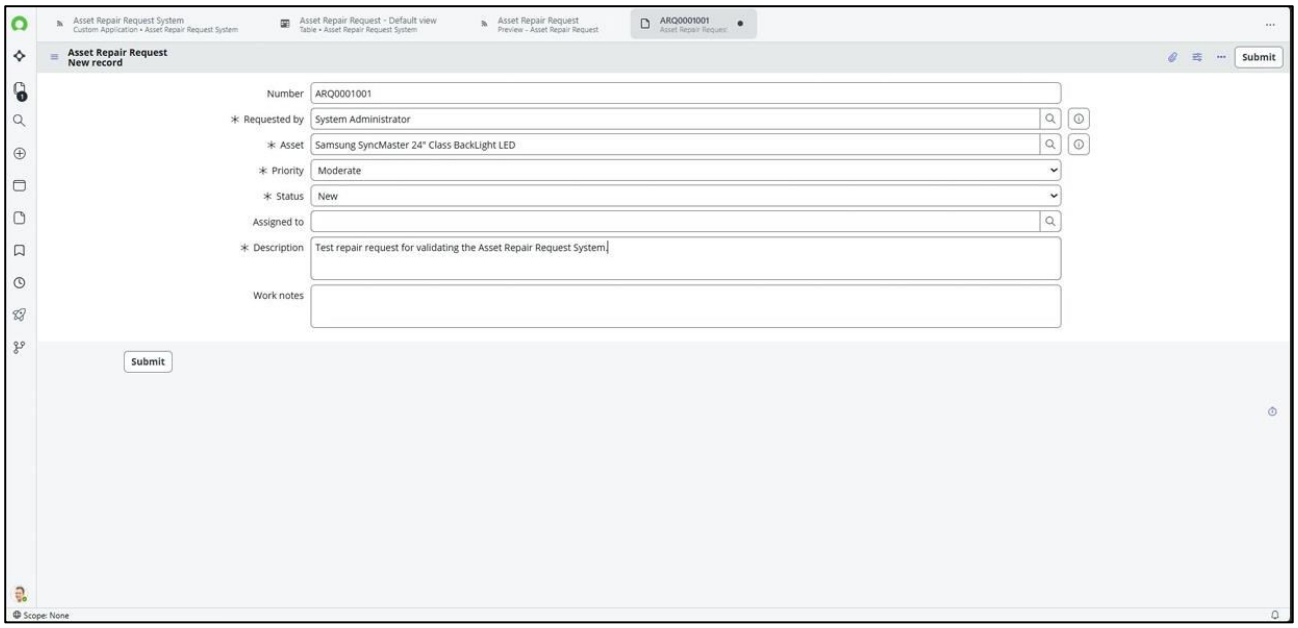


Figure 4.11 Repair Request Creation

4.4.3 Workflow Execution and Activation

The configured flow was tested to verify that the record-created trigger and assignment action executed. The flow test completed successfully, and the flow was activated after testing.

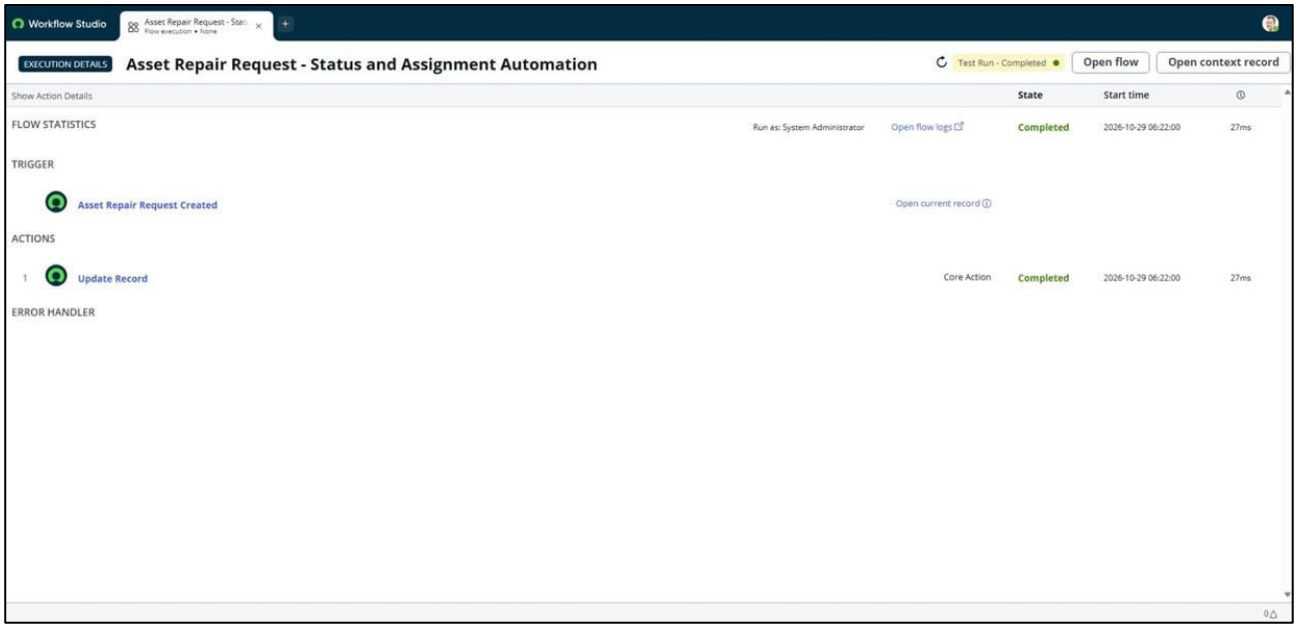


Figure 4.12 Successful Flow Execution

4.5 Results and Interpretation

The implementation demonstrates the principal functions of the Asset Repair Request System: structured request entry, ARQ request numbering, role-based access configuration, and automatic

assignment to the Service Desk. The observed behaviour aligns with the functional requirements defined during the design phase.

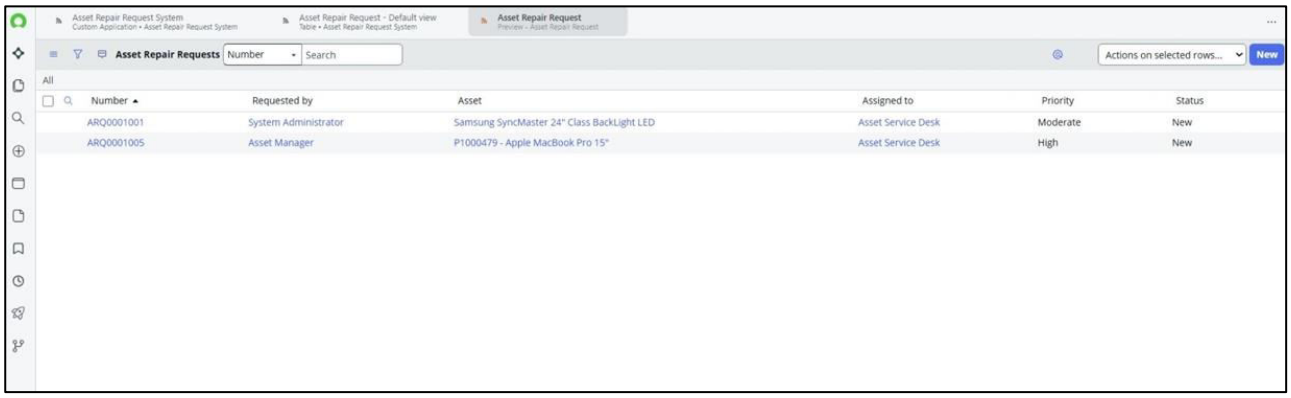


Figure 4.13 Asset Repair Request List View

The observed results support the feasibility of implementing a basic asset repair request process using ServiceNow's application development and workflow capabilities. The results should be interpreted alongside the limitations identified in the following section.

4.6 Limitations and Validation

Several limitations remained at the end of the implementation and testing process:

- Publishing the application to the ServiceNow App Repository was unsuccessful because of a vendor-information or connectivity error.
- The attempt to link the application to a Git repository encountered a compatibility error.
- The employee ownership-based read restriction was not conclusively verified under impersonation.
- The Work notes field did not behave as expected during testing, and its persistence issue remained unresolved.

These limitations identify areas for further development and validation. The demonstrated functions provide a working academic implementation, but the unresolved access-control and field-persistence issues should be addressed before the application is considered ready for production deployment.

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Summary of Completed Work

A scoped ServiceNow application named Asset Repair Request System was created using ServiceNow Studio. The application provides a dedicated environment for managing asset repair requests and includes three application roles: Employee, Service Desk, and Admin. The primary data structure is the Asset Repair Request table, which stores the core information required for each request. Fields configured on the table include Requested by, Asset, Assigned to, Priority, Status, Description, and Work notes. These fields support structured data entry and subsequent tracking by the Service Desk.

Automatic request numbering was configured on the Asset Repair Request table using the prefix ARQ, a starting number of 1000, and a seven-digit format. This configuration generates unique identifiers such as ARQ0001001 for every new record. A request form was arranged to present the fields in a logical order, and an application module was added to the navigator so that authorized users can access the list and form directly from the ServiceNow application menu.

Role-based access control was implemented through Access Control Lists. The Employee role is intended for request submission, the Service Desk role for operational handling, and the Admin role for administrative management. Table-level permissions were assigned according to these roles. A Flow Designer flow named Asset Repair Request – Status and Assignment Automation was created with a record-created trigger. The flow updates the Assigned to field of a newly created request with the configured Service Desk user, thereby automating the initial assignment step.

Functional testing was performed to verify request creation, record saving, automatic numbering, mandatory-field behaviour, and execution of the assignment flow. Administrative operations such as viewing and updating records were also exercised. The implementation has been documented through screenshots of the application, table configuration, form layout, flow design, and test results, together with a user manual that describes the steps for employees and Service Desk personnel. Certain aspects, including conclusive verification of employee ownership-based read restrictions and the persistence behaviour of Work notes, remain outstanding and are recorded as limitations. These outstanding items do not prevent the application from functioning for demonstration purposes but should be addressed before any production consideration.

5.2 Achievement of Objectives

The project objectives were evaluated against the implemented functionality. Centralized storage of repair requests was achieved through the dedicated Asset Repair Request table, which holds all request records in a single, searchable location within the scoped application. Structured request submission is supported by the configured form, which presents the required fields in a logical sequence and enforces mandatory-field validation where defined. Unique request identification is provided by the ARQ-prefixed auto-numbering configuration, which generates a distinct identifier for every new record.

Role-based access configuration was implemented by defining Employee, Service Desk, and Admin roles and assigning corresponding table-level permissions through Access Control Lists. Automatic assignment of newly created requests is performed by the Flow Designer flow, which updates the Assigned to field upon record creation. Application navigation is provided by the module added to the navigator, allowing authorized users to open the list and form. Functional testing confirmed that records can be created and saved, that numbers are generated, and that the assignment flow executes. Documentation of the implementation is available through the screenshots and the user manual.

The employee ownership-based read restriction, which would limit employees to viewing only their own requests, has not been conclusively verified during the available testing window. Consequently, while role-based configuration is in place, it cannot be stated that all security objectives have been fully achieved. The remaining objectives related to request creation, numbering, assignment, and navigation were met within the scope of the academic demonstration.

5.3 Conclusions

The project demonstrates that a low-code platform such as ServiceNow Studio can be used to construct a purposeful service-management application when the design is kept clear and the official development sequence is followed. The Asset Repair Request System was built by configuring a scoped application, a dedicated table, a form, roles, Access Control Lists, and a Flow Designer flow. These elements work together: the table provides the data structure, the form supports data entry, the roles and ACLs control access, and the flow automates the initial assignment of new requests to the Service Desk.

Centralizing repair-request information in a single table improves visibility for the Service Desk and reduces reliance on informal communication channels. Automating the initial assignment step removes a routine manual task and ensures that every newly created request is directed to the

designated handler. The practical value of these measures lies in providing a consistent and traceable process for recording and managing asset repair requests within an academic demonstration environment.

Working on the project provided experience in application configuration, form layout, role definition, ACL setup, and Flow Designer testing. It also highlighted the importance of validating requirements against actual platform behaviour and of distinguishing between features that have been implemented and those that still require verification. The project has reached a functional state suitable for academic demonstration and further testing, but it is not presented as a production-ready enterprise solution.

5.4 Limitations

Several limitations were identified during implementation and testing. The employee ownership-based read restriction, which is intended to ensure that employees can view only their own requests, has not been conclusively verified. Additional ACL testing is required before this behaviour can be confirmed. The persistence of entries made in the Work notes field was also not successfully confirmed during the available testing window. Until this behaviour is resolved, reliance on Work notes for a complete audit trail remains uncertain.

The Flow Designer flow automates only the initial assignment of a newly created request. It does not implement a complete automated status lifecycle. Status changes continue to be performed manually by the Service Desk. This limited scope was a deliberate design choice for the academic demonstration, but it means that further workflow actions would be needed for more advanced automation. Publishing the application through the ServiceNow App Repository was unsuccessful because of a vendor-information or connectivity error. An attempt to link the application to a source-control repository encountered a compatibility error. These distribution and version-control issues affect the portability and maintainability of the application but do not prevent its use within the development instance.

The limitations related to ownership ACL verification and Work notes persistence are unresolved functional questions. The issues with repository publication and source-control integration are implementation constraints that do not render the application unusable for its intended demonstration purpose. Addressing the verification and persistence items is advisable before any consideration of production use. Resolving these points would also strengthen confidence in the completeness of the access-control and audit-trail behaviour.

5.5 Future Enhancements

Several enhancements can be considered for future development. The first priority is to verify and strengthen record-level security so that employees can access only the requests they are authorized to view. This requires conclusive testing of the ownership-based ACL and any necessary adjustments. Reliable handling of Work notes should also be investigated and corrected so that notes entered by the Service Desk are retained as expected. These two items address unresolved aspects of the current implementation and should be completed before more advanced functionality is added.

- Integration with the Configuration Management Database for accurate asset selection
- Email or Microsoft Teams notifications on status changes
- A Service Portal or Experience page for employee self-service
- SLA definitions and reporting dashboards
- Conclusive testing of ownership-based ACLs
- Resolution of Work notes persistence
- Successful source-control linkage and application packaging

The workflow can be extended to support defined status transitions and additional assignment rules. Such extensions would allow the system to move a request through stages automatically when specified conditions are met, reducing the manual effort currently required of the Service Desk. Email notifications for request creation, assignment, and status changes would improve communication with both the requester and the assigned handler.

Dashboards and reports can be developed to display request counts, priorities, statuses, and pending work. These visual summaries would help the Service Desk monitor workload and identify bottlenecks. Service-level tracking, response-time monitoring, and escalation rules could also be introduced to support more formal service commitments. Asset inventory integration would allow more accurate asset selection when a request is raised and would enable reporting that links repair activity to specific assets.

The deployment and source-control compatibility issues should be resolved to improve the portability and maintainability of the application. Successful packaging and version control would make it easier to move the application between instances and to track configuration changes over time. Security verification and reliable record handling remain the immediate priorities before these more advanced capabilities are pursued.

REFERENCES

1. ServiceNow (2024). ServiceNow Studio and App Engine documentation. Retrieved from https://www.servicenow.com and https://developer.servicenow.com
2. ServiceNow (2024). Flow Designer product documentation. ServiceNow Docs.
3. ServiceNow (2024). Access Control List (ACL) best practices. ServiceNow Docs.
4. Faculty-supplied document (2025). ServiceNow App Engine Studio: Standard Project Creation Guide.
5. ITIL Foundation (AXELOS). ITIL 4 edition – service request and incident management practices.

APPENDIX-1

SCREENSHOT GALLERY

This appendix presents selected screenshots captured during the development, configuration, and testing of the Asset Repair Request System on the ServiceNow platform. The screenshots provide visual evidence of the application's implementation and support the explanations presented in Chapters 3 and 4.

The screenshots cover the following stages of development:

Application Setup: Creation of the scoped application, configuration of the application scope and package, role definition, and ServiceNow Studio overview.

Table and Field Design: Creation of the Asset Repair Request table, configuration of ARQ auto-numbering, definition of core fields, and table permissions.

Form Design: Configuration of the Asset Repair Request form layout for structured data entry.

Access Control: Configuration of UI Policies, Access Control Lists (ACLs), and role-based access permissions.

Functional Testing: Creation and saving of repair requests, list-view verification, impersonation, and administrative update and deletion tests.

Workflow Automation: Configuration of the Flow Designer trigger and Update Record action, execution of flow tests, and flow activation.

Application Navigation: Creation of the application menu and module for accessing repair requests.

These screenshots document the major implementation stages and provide supporting evidence for the design decisions, functionality, and validation activities discussed in this report.

APPENDIX-2

USER MANUAL

A2.1 Employee – Creating a Repair Request

1. Log in to the ServiceNow instance with an account that possesses the employee role.
2. Open the Asset Repair Request System application module.
3. Click New.
4. Complete the mandatory fields: Requested by (normally defaults to the current user), Asset, Description and Priority.
5. Status defaults to New; leave it unchanged.
6. Click Submit. Note the generated ARQ number for future reference.

A2.2 Service Desk – Updating a Request

7. Log in with a service_desk or admin account.
8. Open the Asset Repair Requests list.
9. Select the required record.
10. Update Status (In Progress, On Hold, Completed or Closed) and add Work notes as necessary.
11. Adjust Assigned to if reassignment is required.
12. Save the record.

A2.3 Administrator

Administrators possess full access to all records and to the application configuration artefacts (tables, fields, roles, flows and modules). Routine operational use should be performed with the service_desk role; the admin role is reserved for configuration changes.

ACHIEVEMENTS

The project successfully delivered a working ServiceNow scoped application that demonstrates:

- End-to-end request creation with automatic ARQ numbering
- Role-based separation of employee and Service Desk responsibilities
- Flow Designer automation for initial assignment
- A clean, usable form and consistent status model
- Documented testing of the principal functions

The work satisfies the academic requirement for a substantial ServiceNow project while remaining honest about the limitations that remain to be addressed.

GITHUB REPOSITORY

The source code and project documentation for the Asset Repair Request System are available in the GitHub repository linked below.

Repository Link: https://github.com/Yash-Clash/servicenow-asset-repair-request-system

